Data-product design and contracts

Source-to-target mapping (STTM)

- An STTM describes how data from source tables becomes data in a target table. It identifies the sources and target and, most importantly, specifies the **transformation rules**.
- Example: sources `A` and `B` are joined to create target `T`; a rule can specify whether a source column such as `A1` is copied directly into a target column or transformed.
- Textbook transformation categories discussed:
  - **Direct**: Copy a source column directly to the target.
  - **Constant-based**: Populate a target value with a constant.
  - **Lookup**: Find a value in another table.
  - **Aggregate**: Calculate a value such as a mean.
  - **Derive/transform**: Apply an operation to source data; concatenation may be treated as a subtype.
- Catalyst may use a product-specific taxonomy, with derived transformations split into more detailed categories.
- English-language rules need enough precision to resolve ambiguity, especially around edge cases and column-specific meanings of nulls, zeroes, and minimum or maximum values.
- A design document should clarify such rules before implementation.

Product documentation

- Data-product documentation may include a PRD, data contract, glossary or data dictionary, STTM, and service-level expectations.
- Markdown can be edited in a Markdown editor or rendered as a self-contained HTML document. AI can be asked to add an overview, table of contents, diagrams, visuals, and explanatory prose.
- The demonstration showed that an HTML version can be easier to navigate and share than raw Markdown, and can be iterated on. Model choice involves a

speed and quality trade-off.

Service-level concepts

SLI, SLO, and SLA

- **SLI (Service Level Indicator)**: A measurable indicator of service or data-product performance.
- **SLO (Service Level Objective)**: The desired target for an indicator.
- **SLA (Service Level Agreement)**: The agreed permissible departure from an objective and the consequences or commitments associated with breaching it.
- Examples of indicators discussed:
  - Latency, throughput, and recovery time.
  - Availability and delivery delay.
  - Data completeness, accuracy, freshness, and other data-quality measures.
  - Support response or turnaround time, including a possible duty to explain anomalous data.
- Objectives should be clearly defined. For example, measuring delay relative to a delivery time avoids treating a delivery that is only marginally late as categorically different without specifying a tolerance. Time zone and other delivery-time details may also matter.
- Recovery may mean restoring a service, restoring data, or returning service to an agreed level.

Availability and resilience

- Availability is commonly expressed as a percentage of time or in “nines.” The lecture used these rough annual downtime approximations:
  - 90% availability: about 36 days.
  - 99%: about 3.65 days.
  - 99.9%: about 8 hours.
  - 99.99%: about 1 hour.
  - 99.999%: about 5 minutes.
- These are approximate illustrations; exact downtime depends on the

measurement period.
- Availability can be improved through redundant infrastructure, cloned data centers, load balancing, and failover. Sending a small portion of traffic to secondary systems can help verify that disaster-recovery systems work.
- The lecture noted that contractual service credits or support may apply when a provider misses its commitments, and that spare capacity can affect the availability a provider can offer.

Docker and development environments

Containers and images

- Docker packages an application and many of its dependencies into a **container** to support repeatable environments.
- A **Dockerfile** defines how to build a Docker **image**; an image is a template used to create containers. Dockerfiles commonly begin with `FROM` to extend a base image.
- Dockerfile instructions discussed included:
  - `COPY` to place files in the image.
  - `RUN` to execute build-time commands.
  - `USER` to select the user.
  - `EXPOSE` to document a port.
  - `CMD` or `ENTRYPOINT` to configure what runs when a container starts.
- Containers generally share the host operating-system kernel rather than bundling a complete operating system as a virtual machine does. The lecture contrasted this with virtual machines, which provide a separate operating-system environment.
- Containers can isolate applications and control their network access, ports, and resource use, including CPU and RAM. Resource limits can prevent a faulty or overloaded application from consuming all resources on a shared host.
- **Kubernetes** can manage multiple containers and their interactions across a system.

Dev container

- The VS Code dev container provides a command-line environment separate from the host laptop. Files and tools available in the container can be used from its terminal.
- The course environment is based on a standard data-product container and includes tools for working on data products, including dbt.
- The container's Dockerfile extends a base image and configures dependencies, installation scripts, files, and environment variables. Learners were told that maintaining this provided environment is the platform provider's responsibility.
- The Docker introduction challenges were intended to cover checking the Docker version, running and inspecting a container, writing a Dockerfile, building an image, and creating a container from it.

The paved path and its technologies

Roles of the main components

- The **paved path** is the recommended standardized technology set for creating discoverable data products. A specific use case may justify going off the paved path.
- **Amazon S3**: Cloud object/file storage for the underlying data.
- **Databricks**: Compute that runs transformations against the stored data.
- **dbt**: SQL-based transformation and testing layer.
- **Airflow**: Workflow orchestration, including scheduling and triggering dbt work.
- **Astronomer**: Managed Airflow service, reducing the need for teams to operate Airflow infrastructure themselves.
- **Docker**: Packages and runs the development environment and its dependencies.
- **Git and GitHub**: Version-control code; code changes and pull requests can trigger release and deployment processes.
- **Catalyst**: Supports the data-product workflow, including generating or

assisting with artefacts.
- The intended high-level flow is to transform incoming data into consumer-aligned output. dbt rules run on Databricks, with Airflow determining when workflows run.

Databricks in the course environment

- In the paved-path workflow, most development happens in files in the data-product container rather than in the Databricks UI.
- After the environment is set up and authenticated, dbt work can create tables in a training schema visible in Databricks.
- Databricks provides a catalog through which datasets can be organized and discovered. The lecture described source- and domain-oriented catalogs as part of the broader environment.
- Data-product code is committed to Git, with supporting deployment machinery handling release. The UI is mainly a compute and catalog view for this workflow, rather than the primary place to write code.

dbt and Airflow

dbt

- dbt was described as an **ELT** tool: data is loaded first, and transformations are then performed in the data platform.
- dbt projects primarily use SQL, with Jinja-style templates such as `{{ ... }}` that are rendered into SQL.
- A dbt SQL file defines a **model**, which may materialize as a table, view, or another configured form. The file name typically determines the model name.
- dbt executes transformations that create or update the resulting relations; developers often write `SELECT` queries that define the model rather than hand-writing all the resulting table-creation and insertion operations.
- YAML files can configure models and tests, including materialization choices such as tables or views. Whether an intermediate model should be a table or a view depends partly on downstream consumers.

- Commands mentioned included `dbt debug`, `dbt seed`, `dbt run`, `dbt test`, and `dbt build`. `dbt seed` can load local data, such as CSV files, into the platform.

Airflow

- Airflow workflows are expressed as **DAGs**—directed acyclic graphs of tasks—with dependencies, branches, or parallel work.
- Airflow tasks are commonly written in Python. The lecture also mentioned YAML-based blueprints as a way to populate standard task templates.
- Databricks operators can run Databricks work. A dbt task group can represent dbt project stages as Airflow tasks.
- Airflow handles the timing and triggering of work, such as running dbt on a schedule or in response to an event.

Data systems: concepts and lecture framing

Data stores, databases, and relational data

- A **data store** or file system can hold files without requiring them to share a common schema. This makes the querying layer responsible for understanding each file's structure.
- A **database** provides more structure. A relational database organizes data into tables (relations).
- The lecture described a relation mathematically as a set of tuples: rows are unordered, while the values within a row have defined columns.
- **ACID** properties were discussed: atomicity, consistency, isolation, and durability. The lecture connected transactions to reliable operations such as debiting one account and crediting another as a single unit.
- **OLTP** systems focus on transactional operations and consistency; **OLAP** systems focus on analysis and bulk reading or transformation. dbt was discussed in the context of ELT and analytical workloads.
- The lecture's historical account of non-relational systems and distributed data technologies was a simplified teaching narrative, not a comprehensive evaluation of those systems.

Data lakes and lakehouses

- A **data lake**, as presented in the lecture, combines cloud file storage with a query-processing layer.
- The lecture used Hadoop as an example of technologies for working with large distributed file systems and Spark as a later query and processing engine.
- A **lakehouse** was described as combining cloud-scale file storage with relational, table-oriented access and transaction-like guarantees.
- In the paved-path example, S3 provides storage and Databricks provides compute. Delta-format data and its metadata or logs help manage table state and support reliable operations over files.
- The lecture's simplified description was that a lakehouse brings database-like properties to data stored in cloud object storage.

Session outline recorded in the notes

- 09:00–09:20: STTM, SLA/SLO/SLI, and service availability and performance.
- 09:20–09:40: Learner work on STTM and service-level examples.
- 09:45–10:00: Break.
- 10:05–10:15: Demonstration using a group's output.
- 10:15–10:30: Overview of the paved path.
- 10:30–10:50: Databricks.
- 10:50–11:05: Break.
- 11:05–11:26: Docker.
- 11:26–12:00: Docker learner challenge.
- 12:00–13:00: Lunch.
- 13:05–13:20: dbt and Databricks.
- 13:20–14:15: Learner exploration of dbt code using Catalyst.
- 14:10–14:30: dbt debrief.